

Phase 1 Recap & Mini System

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles: Putting It All Together

English:

If any single chapter in Phase 1 felt shaky while reading this recap, that's useful signal, not failure -- go back to that specific chapter before Phase 2, since Phase 2 assumes every Phase 1 mental model is already load-bearing.

Hindi:

अगर यह recap पढ़ते हुए Phase 1 का कोई chapter कमज़ोर लगा, तो यह एक उपयोगी संकेत है, कोई असफलता नहीं -- Phase 2 शुरू करने से पहले उस specific chapter पर वापस जाएं, क्योंकि Phase 2 यह मानकर चलता है कि Phase 1 के हर mental model पर आप पहले से पूरी तरह भरोसा कर सकते हैं।

Important Words:

- **Load-bearing (figurative)** – मज़बूत आधार वाला

English:

This chapter has no new concepts. Its job is to force you to combine everything from Chapters 1-14 into one coherent design, the way an actual interview would. We'll design a Pastebin-style service.

Hindi:

इस chapter में कोई नया concept नहीं है। इसका काम है आपको Chapters 1-14 की हर चीज़ को मिलाकर एक coherent design बनाने पर मजबूर करना, बिल्कुल जैसे असली interview में होता है। हम एक Pastebin जैसी service design करेंगे।

Important Words:

- **Coherent** – सुसंगत / तालमेल वाला

Step 1 -- Clarify requirements

English:

- **Functional:** a user can paste text and get a short URL back; anyone with the URL can view the paste.
- **Non-functional:** reads will vastly outnumber writes; the service should feel instant; should survive a single server failing.

Hindi:

- **Functional:** एक user text paste करके एक short URL पा सकता है; URL रखने वाला कोई भी उस paste को देख सकता है।
- **Non-functional:** reads, writes से कहीं ज़्यादा होगी; service instant महसूस होनी चाहिए; एक server fail होने पर भी system बना रहना चाहिए।

Step 2 -- Back-of-the-envelope estimation

English:

Assume 1 million new pastes/day, and a generous 20:1 read:write ratio. Write QPS ~10/sec. Read QPS ~200/sec. Average paste size ~2KB, so daily storage ~2GB/day, ~730GB/year -- entirely manageable on a single database.

Hindi:

मान लें रोज़ 10 लाख नए pastes बनते हैं, और read:write ratio 20:1 है। Write QPS ~10/sec। Read QPS ~200/sec। हर paste का average size ~2KB, तो daily storage ~2GB/day, ~730GB/year -- यह एक ही database पर आसानी से संभल सकता है।

Step 3 -- High-level architecture

English:

Client -> DNS -> Load Balancer -> stateless application servers -> a relational database storing paste metadata. Because reads dominate 20:1, we put a cache in front of the database for paste lookups -- the single highest-leverage addition.

Hindi:

Client -> DNS -> Load Balancer -> stateless application servers -> एक relational database जो paste की metadata रखता है। चूंकि reads 20:1 से हावी हैं, हम paste lookups के लिए database के आगे एक cache लगाते हैं -- यह सबसे ज़्यादा फायदेमंद बदलाव है।

Important Words:

- **Dominate** – हावी होना
- **Leverage (figurative)** – फायदा / प्रभाव

Step 4 -- The core algorithm: generating short codes

English:

A simple, defensible approach: auto-increment an ID in the database, then encode that integer in base-62 to produce a short string. HLD interviewers expect you to at least name the approach.

Hindi:

एक सरल, defend करने लायक तरीका: database में एक auto-increment ID रखें, फिर उस integer को base-62 में encode करके एक छोटा string बनाएं। HLD interviewers कम से कम इतनी उम्मीद करते हैं कि आप approach का नाम तो बताएं।

Step 5 -- Sanity-check against non-functional requirements

English:

Before calling the design done, hold it against the non-functional requirements from Step 1. Availability: three stateless app servers behind a load balancer means losing one doesn't take the system down.

Hindi:

design को पूरा मानने से पहले, इसे Step 1 की non-functional requirements के सामने रखकर जांचें।

Availability: load balancer के पीछे तीन stateless app servers होने का मतलब है कि एक के जाने से पूरा system नहीं गिरगा।

2. Real-World Example / Case Study

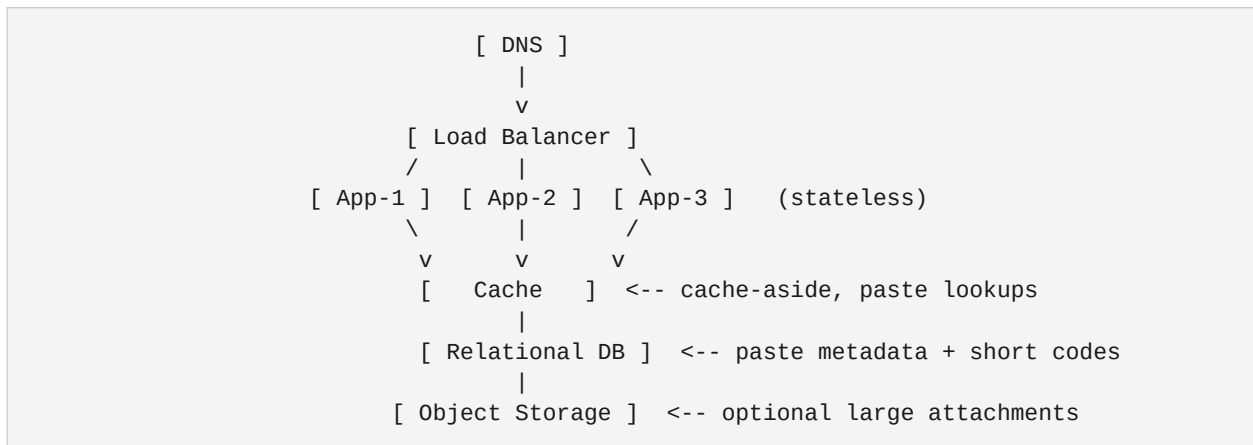
English:

This exact shape -- write-light, read-heavy, cache-in-front-of-a-relational-store -- describes a huge fraction of real consumer systems: Flipkart's product listing pages, Swiggy's restaurant menu pages, and PhonePe's merchant directory all share this profile.

Hindi:

यही shape -- कम writes, ज़्यादा reads, relational store के आगे एक cache -- बहुत सारे असली consumer systems में देखिती है: Flipkart के product listing pages, Swiggy के restaurant menu pages, और PhonePe की merchant directory, यह सब इसी profile को follow करते हैं।

3. Diagram (English labels kept as-is)



4. Trade-offs Table (Reference Table – English)

Decision made	Alternative considered	Why this choice, at this scale
Relational DB for paste metadata	Document store	Simple, structured records; ACID not critical but no downside
Cache-aside in front of DB	No cache	20:1 read:write ratio makes caching the highest-leverage change
Single DB instance (no sharding yet)	Sharded DB from day one	~730GB/year fits comfortably in one instance
Base-62 encoded auto-increment ID	Random string + collision check	Simpler, no collision-retry logic needed

5. Common Interview Questions

Q1: "Walk me through your architecture for this system, end to end."

English:

Model approach: Follow the same order used in this chapter -- requirements, estimation, then architecture, narrating trade-offs at each step.

Hindi:

Model approach: इस chapter जैसा ही क्रम follow करें -- requirements, estimation, फिर architecture, हर step पर trade-offs बताते हुए।

Q2: "What's the very first thing that breaks if traffic grows 100x?"

English:

Model approach: Reason from the estimation numbers -- at 100x, write QPS goes to 1000/sec and yearly storage to ~73TB, at which point the single relational database becomes the bottleneck first.

Hindi:

Model approach: estimation के numbers से सोचें -- 100x पर, write QPS 1000/sec हो जाएगा और yearly storage ~73TB, इस स्थिति में सबसे पहले वह single relational database ही bottleneck बनेगा।

6. Practice Exercises (Capstone)

English:

- Without looking back, design this pastebin system from scratch: requirements, estimation, architecture diagram, and 2 trade-offs.
- Now add a new requirement -- pastes can be private and password-protected -- and explain what changes.
- Using your own estimation numbers, identify at what scale this design would start to strain, and name the component you'd address first.

Hindi:

- बना पीछे देखे, इस pastebin system को शुरू से design करें: requirements, estimation, architecture diagram, और 2 trade-offs।
- अब एक नई requirement जोड़ें -- pastes private और password-protected हो सकते हैं -- और बताएं कि design में क्या बदलेगा।
- अपने estimation numbers से बताएं कि किस scale पर यह design दबाव में आएगा, और सबसे पहले कौन सा component ठीक करेंगे।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"System Design for Beginners: A Complete Roadmap"	Good sanity check of Phase 1 coverage.
English	"Complete System Design Roadmap for 90 Days FAANG Interviews"	Shows where Phase 1 fits in a full timeline.
Hindi	"Complete System Design Roadmap 2025" — CodeHelp	Full Hindi roadmap, worth revisiting now.
Hindi	"System Design Roadmap for Beginners" (Hindi & English)	Reinforces the fundamentals-first structure.

8. Key Takeaways

English:

- A repeatable order works for any HLD problem: requirements -> estimation -> architecture -> deep dive -> trade-offs.
- Back-of-the-envelope numbers should directly justify each architecture decision.
- Read-heavy, write-light systems almost always benefit most from caching first.
- Don't reach for sharding, multi-region, or microservices until your numbers show you need them.
- This write-light/read-heavy/cache-in-front-of-relational-DB shape reappears across most consumer products.
- You are now ready for Phase 2, which scales this exact design past what a single database can handle.

Hindi:

- किसी भी HLD problem के लिए एक दोहराया जा सकने वाला क्रम काम करता है: requirements -> estimation -> architecture -> deep dive -> trade-offs।
- Back-of-the-envelope numbers को हर architecture decision को सीधे justify करना चाहिए।
- Read-heavy, write-light systems को लगभग हमेशा सबसे पहले caching से फायदा होता है।
- जब तक numbers ना दिखाएं कि ज़रूरत है, तब तक sharding, multi-region, या microservices की तरफ मत जाएं।
- यह write-light/read-heavy/cache-in-front-of-relational-DB shape ज्यादातर consumer products में देखिती है।
- अब आप Phase 2 के लिए तैयार हैं, जो इसी design को उस बढ़ि से आगे scale करता है जहाँ एक database काफी नहीं रहता।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Coherent	सुसंगत / तालमेल वाला	Logically connected and consistent
Dominate	हावी होना	To be much larger or more significant
Leverage (figurative)	फायदा / प्रभाव	The advantage gained from an action
Load-bearing (figurative)	मज़बूत आधार वाला	Something the rest depends on, like a supporting wall